

Kubernetes v1.34 Installation

I once sat in front of a blank Ubuntu server.

No containers. No cluster. Just a blinking cursor and a massive microservices architecture I was determined to deploy from scratch.

So I did what most engineers do the first time.

I ran `kubeadm init` almost immediately.

It failed. Obviously. That Failure initiates learning Self-Managed K8s.

Here's what that failure taught me — the why behind every Kubernetes prerequisite:

The First Lesson: Kubernetes doesn't trust RAM unless it's the REAL RAM.

Before anything else, disable swap.

Not because some documentation says so. But because Kubernetes runs an agent on every node called **Kubelet** — and Kubelet's entire job is to look at a machine and say, *"This node has X memory, I can schedule Y pods here."*

When swap is ON, the OS quietly starts using disk as fake RAM. Kubelet doesn't know this. It reports a number that *feels* real but isn't. Pods get scheduled beyond the machine's true capacity.

And then one day, under load, the Linux kernel panics — it starts killing processes to survive. That's **OOMKill** — the Out-Of-Memory Killer. A container gets terminated mid-flight. Exit Code 137. Status: OOMKilled.

r cluster didn't crash because of bad code. It crashed because the foundation lied about how much memory it had.

Two commands fix this forever:

`sudo swapoff -a` (turns it off now)

`sudo sed -i 's/ swap / s/^\(.*\)$/#\1/g' /etc/fstab` (ensures it never comes back after reboot)

The Second Lesson: The kernel needs to be taught to *watch* the network.

Next, load two kernel modules — overlay and br_netfilter.

Think of overlay as the reason containers can share a base Linux image and still have their own writable layer on top. It's what makes a 500MB image not copy itself 50 times across 50 containers.

And br_netfilter? Imagine a server as a building, and containers as rooms. When a packet of data travels from one room to another, br_netfilter makes sure the security guard (iptables) at the building entrance actually sees that traffic — not just external visitors, but internal ones too.

Without it, a CNI plugin is shouting directions into a void. Calico, Flannel, Canal — they all rely on iptables being in the loop.

also flip on **IP forwarding** — because Kubernetes nodes need to *route* traffic, not just receive it. A node that can't forward packets is like a post office that accepts mail but never delivers it.

The Third Lesson: Someone has to actually *run* the containers.

This is where **Containerd** enters.

People confuse Docker and Containerd. Docker is a developer experience — a friendly interface. Containerd is the engine underneath — the thing that actually pulls images, creates namespaces, mounts filesystems, and starts processes.

Kubernetes talks to Containerd through the **CRI** (Container Runtime Interface) — a standardised handshake. When a pod needs to be created, Kubernetes says "*spin up this container*" to Containerd, and Containerd makes it happen.

install it from the Docker repository (because that's where containerd.io lives), generate a default config, and make one critical change — ensure it uses the **systemd cgroup driver**, so it speaks the same resource management language as the Linux system it runs on.

Without this alignment, resource limits on pods become suggestions, not rules.

The Fourth Lesson: Three tools, three different relationships with the cluster.

kubeadm — the architect. It builds the cluster. Generates TLS certificates for every component, bootstraps the Control Plane, and hands a join command for r workers.

kubelet — the heartbeat. It runs on every single node, forever. It watches the API Server for pod assignments, tells Containerd to start them, and reports back. It is the nervous system.

kubectl — User's voice. The CLI that lets *speak* to the cluster. Deploy an app, check logs, describe a failing pod — all through kubectl.

lock their versions with apt-mark hold because Kubernetes components are famously sensitive to version mismatches. A kubelet one minor version ahead of the API server? Undefined behaviour. Version discipline is survival.

The Fifth Lesson: kubeadm init is not magic — it's choreography.

When finally run

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

, something beautiful and precise happens:

→ PKI certificates are generated for the API Server, etcd, and every control plane component → Static pod manifests are written to /etc/kubernetes/manifests/ — the API Server, Controller Manager, Scheduler, and etcd wake up as pods managed directly by Kubelet → CoreDNS is installed so pods can resolve each other by name → kube-proxy is installed so service IPs route correctly to pod IPs → A **join token** is printed — the key worker nodes use to enter the cluster

copy the admin config to ~/.kube/config, chown it to rself, and for the first time — kubectl get nodes actually responds.

One node. But Status: NotReady.

It's not ready yet because pods still can't talk to each other. The cluster has no network fabric.

The Sixth Lesson: Choose CNI plugin like choosing infrastructure philosophy.

apply Flannel — a simple YAML file from GitHub — and watch it deploy as a **DaemonSet** across every node.

Flannel wraps pod traffic in VXLAN tunnels. Every node gets a subnet. Packets travel across nodes like they're on the same LAN, even though they're not. Simple, reliable, perfect for learning and smaller environments.

But Calico is the one to choose in production. It operates at **Layer 3**, uses **BGP** to route packets natively — no encapsulation overhead. And most importantly, it lets write **Network Policies** — firewall rules *inside* cluster. Which pod can talk to which pod. Which namespace is isolated. Real security posture, not just connectivity.

Canal sits between them — Flannel's simplicity, Calico's policy engine. The pragmatist's choice.

The Final Lesson: Worker nodes aren't followers — they're equals in discipline.

Every worker node demands the same respect: swap off, kernel modules loaded, IP forwarding on, Containerd configured, kubelet installed.

Only then do run the kubeadm join command — presenting the token and certificate hash like credentials at a secure door. The Control Plane validates, registers the node, and Kubelet starts pulling pod assignments.

kubectl get nodes — and now see them. All of them. Ready.

There's something deeply satisfying about a Kubernetes cluster coming to life — not because followed steps, but because understood *why* each step existed.

The cluster isn't magic. It's just a very disciplined system that demands be disciplined too.

Kubernetes:

Installation – Version -1.34

Disable swap:

Kubernetes relies on Kubelets.

Kubelets determines a nodes available resources(CPU & Memory), helps while scheduling PODs.

Kubelet is designed to base its Scheduling Decisions on PHYSICAL MEMORY.

If Swap is Present , Kubelet gets inaccurate memory capacity leading to Over Commitment & unpredictable behaviour.

Kubernetes has OOMKill Mechanism , Swap prevents this from working as intended , affects stability.

[OOMKill – Kubernetes's Event that signifies a container has been terminated by the Linux Kernel's Out-Of-Memory(OOM) Killer because it exceeded its Allocated Memory Limit.]

[OOMKill identification → Run “kubectl describe pod <pod name>” → Exit Code 137 , Status OOMKilled. Remedy → Increase memory/fix memory leak using monitoring tools]

SWAP Slower than RAM, latency Spikes.

CLI to Disable SWAP:

```
sudo swapoff -a          #Disables all Swaps, Temporary – enabled after Reboot.
```

```
sudo sed -i ' / swap / s/^(.*)$/#\1/g' /etc/fstab # Stream Editor used to edit /etc/fstab , Permanent.
```

Enabling Kernel Modules & IP Forwarding:

```
sudo modprobe overlay # Loads Overlay module , Essential to stack image layers , sharing common files.
```

```
sudo modprobe br_netfilter # Loads Bridge Netfilter module, Essential to pass traffic to IPTables , Firewalls to process.
```

#Most CNI Plugins (like Calico, Flannel, etc) and Kube-Proxy rely heavily on iptables rules for Service , pod to pod routing/communication.

Below lines writes new config file sets kernel parameters (iptables, ipforwarding)

```
printf "%s\n" "net.bridge.bridge-nf-call-ip6tables = 1" "net.bridge.bridge-nf-call-iptables = 1"
"net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/kubernetes.conf
```

applying new network settings to running kernel without reboot

```
sudo sysctl --system
```

*! Before installing Containerd , Run Below Script to Update Docker Repository to system's package sources(/etc/apt/sources.list.d). Containerd.io located hosted in Docker Repository. Else we get Error “E: Unable to locate package containerd.io”

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo \
```

```
"deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu \
```

```
"$(. /etc/os-release && echo "$VERSION_CODENAME)" stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Installing Container Runtime [Ex- Containerd]

[Notations: CNI – Container Network Interface , CRI – Container Runtime Interface]

```
sudo apt update # Update local package index
```

```
sudo apt install -y containerd.io # Installs Containerd – responsible for container lifecycle within pods
```

```
sudo mkdir -p /etc/containerd # Creates config directory , where Containerd config file(config.toml) resides.
```

#Below Command Generates default config and inserting to config.toml, necessary to load SYSTEMD CGROUP DRIVER(used for Resource Management with Host System)

```
sudo containerd config default | sudo tee /etc/containerd/config.toml
```

```
sudo systemctl restart containerd # Restart Containerd Services.
```

```
sudo systemctl enable containerd # Enables on boot
```

Add Kubernetes APT Repository:

```
sudo apt update #Refreshes Local Package Index.
```

```
sudo apt install -y apt-transport-https ca-certificates curl # Necessary utility installation.
```

```
sudo curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o  
/etc/apt/keyrings/kubernetes-apt-keyring.gpg # Download & Install Kubernetes GPG Signing key.
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]  
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ ' | sudo tee /etc/apt/sources.list.d/kubernetes.list #Adds K8s Repo
```

```
sudo apt update # Refreshes Packages again
```

Install kubeadm, kubelet, and kubectl:

[k8s Core Components – kubeadm , kubelet , kubectl]

[kubeadm (Cluster Bootstrap Tool) – initialize the control-plane (master node) & to join worker nodes, automates cluster component setup]

[kubelet (Node Agent) – Primary agent runs on every node , register node to control plane, responsible for managing pods lifecycle , also for containers by communicating with container run time(containerd).]

[kubectl (CLI Tool) – Client Utility(Users) to interact with control plane(Actions: Deploy, inspect, log view).]

Below command installs Core Components mentioned

```
sudo apt install -y kubelet kubeadm kubectl
```

```
sudo apt-mark hold kubelet kubeadm kubectl # Version Locking – k8s has high sensitivity on this
```

Initialize the Control Plane (Master Node Only):

Initializes k8s Control Plane Node (Master Node).

kubeadm init – performs critical actions below

- ~ Generates Security Certificates (for API Server, etcd, etc)

- ~Generates kubeconfig files for administrators and components

- ~Installs Control Plane Components (API Server, Control Manager, Scheduler, etcd) as static Pods in the “/etc/kubernetes/manifests” directory.

- ~Installs CoreDNS and kube-proxy

- ~Prints “kubeadm join” Command for worker nodes.

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 # Use a suitable Pod network CIDR
```

[notes: 10.244.0.0/16 – Standard default CIDR for CNI Plugins like Flannel, Calico]

[Flannel – Creates Simple Overlay Network by , Assigns unique subnets to each nodes, uses encapsulation VXLAN (IP-in-IP), making communication between Pods across nodes (Cluster Wide) as on same local network, used in smaller cluster environments.]

[Calico – Operates at OSI Layer 3 , Uses BGP to route packets between nodes directly, designed for production grade clusters, high performance networking, robust network policy enforcement. Allows defining fine-grained rules to control ingress and egress traffic for pods, namespaces, etc,. enabling strong network segmentations and security.]

[Canal – CNI Plugin – Combines Flannel’s Networking Capabilities & Calico’s Network Policy Enforcement]

We get kubeadm join command for worker nodes.

Setting Up Kubectl (Master Node Only):

```
mkdir -p $HOME/.kube #creates .kube (user’s home directory) , which has configuration file(config).
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config #Copies admin.conf to .kube/config
```

[/etc/Kubernetes/admin.conf – generated after kubeadm init , contains Cluster definition, User Credentials and Security Certificates to authenticate administrator]

[\$HOME/.kube/config – Kubectl uses it automatically as its configuration file]

Changes file permission to Current-Non Root User(UID) and Group(GID)

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Install a Pod Network Add-on (Master Node Only):

#Below command fetch and apply Flannel CNI Plugin

[kube-flannel.yml – Configuration file of Flannel, Deploying Flannel Agent as DaemonSet, Necessary permissions (ServiceAccount & ClusterRole) created so the Flannel Agent reads POD CIDR and update node information to Kubernetes API Server in Master Node, Creates Cross-Node Tunnels-Cluster Wide.]

```
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
```

#Generate kubeadm join command and save in a file.

```
sudo kubeadm token create --print-join-command > k8s-join
```

Join Worker Nodes Using kubeadm join command provided from kubeadm init command output.

Kubernetes Installation Setup is Done , Run “kubectl get nodes” to view nodes and Ready status.

In Worker Nodes:

Before Executing “kubeadm join” setup below prerequisites.

```
sudo swapoff -a
```

```
sudo sed -i ' / swap / s/^(.*)$/#\1/g' /etc/fstab
```

```
sudo modprobe overlay
```

```
sudo modprobe br_netfilter
```

```
printf "%s\n" "net.bridge.bridge-nf-call-ip6tables = 1" "net.bridge.bridge-nf-call-iptables = 1" "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/kubernetes.conf
```

```
sudo sysctl --system
```

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo \
```

```
"deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
```

```
"$(. /etc/os-release && echo "$VERSION_CODENAME)" stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt update
```

```
sudo apt install -y containerd.io
```

```
sudo mkdir -p /etc/containerd
```

```
sudo containerd config default | sudo tee /etc/containerd/config.toml
```



```
sudo systemctl restart containerd
```

```
sudo systemctl enable containerd
```

```
sudo apt update
```

```
sudo apt install -y apt-transport-https ca-certificates curl
```

```
sudo curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o  
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]  
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
sudo apt update
```

```
sudo apt install -y kubelet kubeadm kubectl
```

```
sudo apt-mark hold kubelet kubeadm kubectl
```

```
sudo kubeadm join <ControlPlane-IP>:<Port> --token <Token> --discovery-token-ca-cert-hash  
sha256:<Hash>
```

*! If Error occurs Run “sudo kubeadm reset -f” and Rerun from 1st Step in Worker node to Join Cluster.